Deep Learning Laboratory

External Exam: Theory Questions and Answers for all 6 Practicals

Lab Practice IV (414447), BE (Information Technology) 2019 Course, SPPU

The practical exam is based on your journal. The examiner asks theory and implementation questions on the experiment you performed (syllabus guideline). For each practical this PDF gives: a **snapshot** of your own notebook numbers, **questions with answers** covering the theory and your code/output, and a **Check before the exam** box where your notebook has a problem an examiner could notice.

Contents

Section	Topic
Core	Theory common to every practical (activation, backpropagation, optimizers, regularization, metrics)
Practical 1	Study of Deep Learning Packages (TensorFlow, Keras, Theano, PyTorch)
Practical 2	Feedforward Neural Network with Keras and TensorFlow (MNIST, SGD)
Practical 3	Image Classification with a CNN (MNIST, 4 stages)
Practical 4	Autoencoder for Anomaly Detection (Credit Card Fraud)
Practical 5	Continuous Bag of Words (CBOW) Model
Practical 6	Transfer Learning with a Pre-trained CNN (VGG16, tf_flowers)

Before you go in: the three notebooks with problems are Practical 3 (validation data), Practical 5 (CBOW never trained) and Practical 6 (test set overlaps training set). Read their boxes first.

Core theory asked in every deep learning practical

C1. What is deep learning? How is it different from machine learning?

Deep learning is a subset of machine learning that uses neural networks with many hidden layers to learn features automatically from raw data (pixels, text, audio). Classical ML needs hand-crafted features. Deep learning needs more data and compute (GPU) but scales better.

C2. What is an artificial neuron / perceptron?

It computes a weighted sum of its inputs plus a bias and passes it through an activation function: `y = f(w1x1 + w2x2 + ... + wnxn + b)`. Weights and bias are the learnable parameters.

C3. What is a Multilayer Perceptron (feedforward network)?

Layers of fully connected neurons (input, one or more hidden, output) in which information flows only forward. There are no loops, unlike an RNN.

C4. Name the common activation functions with formula and use.

Sigmoid `1/(1+e^-x)`, range (0,1), saturates, used for binary output. **Tanh** `(e^x-e^-x)/(e^x+e^-x)`, range (-1,1), zero-centred. **ReLU** `max(0,x)`, cheap and the default for hidden layers, can 'die' for negative inputs. **Leaky ReLU** `max(0.01x, x)` fixes dying ReLU. **ELU** `x if x>0 else a(e^x-1)`, smooth for negatives. **Softmax** `e^zi / sum(e^zj)`, turns scores into probabilities that sum to 1 (multi-class output).

C5. Why do we need non-linear activation functions?

Without them, any stack of layers collapses into one linear transformation and the network could not learn complex patterns.

C6. Explain forward propagation and backpropagation.

Forward pass: inputs go layer by layer to produce a prediction and the loss is computed. Backpropagation: the chain rule is applied from the output layer back to the input to compute dLoss/dWeight for every weight. The optimizer then uses these gradients to update the weights.

C7. What is gradient descent? Batch vs stochastic vs mini-batch?

Update rule `w = w - lr * dL/dw`. **Batch** uses the whole dataset per update, **stochastic** uses 1 sample, **mini-batch** uses a small group (Keras default batch size 32). Keras's `SGD` optimizer is really mini-batch gradient descent.

C8. What are learning rate and momentum?

Learning rate is the step size: too large diverges, too small is slow. Momentum keeps a running velocity `v = beta*v - lr*grad; w = w + v`, which speeds up training and damps oscillations (Practical 3 uses momentum 0.9).

C9. Name common optimizers.

SGD, SGD with momentum, RMSprop (per-parameter adaptive step using a moving average of squared gradients), Adam (momentum plus RMSprop; defaults lr 0.001, beta1 0.9, beta2 0.999), Adagrad, Adadelta.

C10. What is the vanishing gradient problem?

In deep networks with sigmoid or tanh, derivatives are below 1 and get multiplied layer after layer, so gradients in early layers become almost zero and those layers stop learning. Remedies: ReLU family, proper initialization (He, Xavier), batch normalization, residual or dense connections, LSTM for sequences. The opposite, exploding gradients, is handled by gradient clipping.

C11. Common loss functions?

MSE `mean((y-y^)^2)` for regression and autoencoders; binary cross-entropy for 2 classes; categorical cross-entropy `-sum(y_k log y^_k)` for one-hot multi-class labels; sparse categorical cross-entropy is the same but takes integer labels.

C12. What is overfitting and underfitting? How do you detect and fix them?

Overfitting: very high training accuracy but much lower validation accuracy (model memorises). Underfitting: poor on both. Fix overfitting with more data, data augmentation, dropout, L1/L2 regularization, early stopping, smaller model. Fix underfitting with a bigger model, more epochs, better features.

C13. Explain regularization: L1, L2, dropout, drop-connect.

L1 adds `lambda*sum|w|` to the loss and drives weights to exactly zero (sparse). **L2** adds `lambda*sum w^2` (weight decay) and keeps weights small. **Dropout** randomly switches off a fraction of neurons during training so neurons cannot co-adapt. **DropConnect** drops individual weights instead of whole neuron outputs. **Early stopping** halts training when validation performance stops improving.

C14. What are hyperparameters? Give examples.

Settings chosen by the user, not learned: learning rate, batch size, number of epochs, number of layers and units, activation, optimizer, momentum, dropout rate, window size and embedding size (CBOW). Parameters (weights and biases) are the learned ones.

C15. Epoch, batch size, iteration?

Epoch = one full pass over the training data. Batch size = samples per update. Iterations per epoch = samples / batch size. Example: 60,000 / 32 = 1,875 (the `1875/1875` in your logs).

C16. Why split data into train, validation and test sets?

Train to learn weights, validation to tune hyperparameters and stop early, test for one final unbiased estimate. The test data must never be seen during training; otherwise it is data leakage and the accuracy is inflated.

C17. Explain accuracy, precision, recall, F1 and the confusion matrix.

Confusion matrix: rows true class, columns predicted class (TP, FP, FN, TN). Accuracy = (TP+TN)/all. Precision = TP/(TP+FP). Recall = TP/(TP+FN). F1 = 2PR/(P+R). With imbalanced data accuracy alone is misleading.

C18. Sequential API vs Functional API in Keras?

Sequential is a plain linear stack of layers (Practicals 2, 3, 6). Functional connects layers like a graph, `Model(inputs, outputs)`, allowing multiple inputs, outputs and branches (Practical 4).

C19. What do compile, fit, evaluate and predict do?

`compile` sets optimizer, loss, metrics. `fit` trains for given epochs and returns a History. `evaluate` returns loss and metrics on given data. `predict` returns model outputs for new inputs.

C20. Why normalize / scale the input?

Keeps features in a similar small range so gradients are well behaved and training converges faster and more stably. Pixels are divided by 255 to fall in [0,1].

Practical 1: Study of Deep Learning Packages (TensorFlow, Keras, Theano, PyTorch)

Snapshot of your notebook

Notebook	DL_Ass__01.ipynb (Anaconda, Python 3.9, Windows)
Versions installed	TensorFlow 2.10.0, Keras 2.10.0, Theano 1.0.5, PyTorch 1.12.1+cpu
Test programs	TF: `tf.reduce_sum(tf.random.normal([1000,1000]))` gave -525.7693. Keras: MNIST loaded, shapes (60000,28,28) and (10000,28,28). Theano: add two scalars, `f(5,7)` gave `array(12.)`. PyTorch: version check

Comparison of the four packages

	TensorFlow	Keras	Theano	PyTorch
Developer	Google Brain (2015)	Francois Chollet (2015); now part of Google	MILA, Univ. of Montreal (2007)	Facebook AI Research (2016)
Level	Low and high level	High-level API	Low-level symbolic library	Low and high level
Graph	Static graph; eager by default in 2.x; `tf.function`	Uses backend graph	Static symbolic graph, compiled to C/CUDA	Dynamic (define-by-run)
Strength	Production: TF Serving, TF Lite, TF.js, TensorBoard, TPU	Fast, simple prototyping	Early pioneer, auto-differentiation	Easy debugging, popular in research
Status	Active	Active (Keras 3 supports TF, JAX, PyTorch)	Development stopped after 1.0 (2017); forks Aesara/PyTensor	Active (PyTorch Foundation)

Questions and answers

Q1. What is TensorFlow? List its features.

An open-source library by Google for numerical computation and machine learning using dataflow graphs: nodes are operations, edges are tensors. Features: eager execution plus `tf.function` graphs, automatic differentiation (`tf.GradientTape`), CPU/GPU/TPU support, TensorBoard visualisation, deployment tools (TF Serving, TF Lite for mobile, TF.js for browser), large model zoo.

Q2. What is Keras and how is it related to TensorFlow?

Keras is a high-level, user-friendly neural-network API (layers, models, optimizers) with minimal code. Since TF 2.0 it is the official high-level API as `tf.keras`; in your notebook Keras 2.10.0 matches TF 2.10.0. Keras 3 is multi-backend.

Q3. What is Theano? Why is it not used much now?

A Python library that defines, optimizes and evaluates mathematical expressions on multi-dimensional arrays using symbolic graphs, compiled to fast C/CUDA code, with automatic differentiation (`T.grad`). It was the base of early Keras. MILA stopped major development after version 1.0 (2017) because TF and PyTorch took over.

Q4. What is PyTorch? What is special about it?

A Python deep learning framework from Facebook AI Research built on tensors with GPU acceleration, `autograd` for automatic differentiation and the `torch.nn` module for building networks. Its dynamic computation graph is built as the code runs, which makes debugging with normal Python tools easy.

Q5. Static vs dynamic computation graph?

Static (Theano, TF 1.x): define the full graph first, then run it; good for optimisation and deployment. Dynamic (PyTorch, TF 2.x eager): graph is built on the fly with normal Python control flow; easier to debug.

Q6. What is a tensor? What are rank and shape?

A tensor is an n-dimensional array. Rank 0 scalar, 1 vector, 2 matrix, 3 and above higher-order. Shape lists the size per dimension, e.g. MNIST train images `(60000, 28, 28)`.

Q7. Explain your TensorFlow test line `tf.reduce_sum(tf.random.normal([1000,1000]))`.

`tf.random.normal([1000,1000])` creates a 2-D tensor of 10^6 random values from a standard normal distribution; `reduce_sum` adds all of them into one scalar. The sum has mean 0 and standard deviation sqrt(10^6) = 1000, so -525.77 is a perfectly normal result.

Q8. Explain your Theano program line by line.

`T.dscalar('x')` declares a symbolic 0-dimensional float64 variable ('d' = double). `z = x + y` only builds a symbolic expression, nothing is computed. `function([x, y], z)` compiles the graph into an executable function. `f(5, 7)` runs it and returns `array(12.)`.

Q9. What is automatic differentiation?

The framework records the operations and applies the chain rule to compute exact gradients automatically: `tf.GradientTape` in TensorFlow, `loss.backward()` / autograd in PyTorch, `theano.tensor.grad` in Theano. It is what makes backpropagation effortless.

Q10. What does the `+cpu` in `1.12.1+cpu` mean? Why GPUs for deep learning?

It is the CPU-only build of PyTorch (no CUDA). GPUs have thousands of cores that do large matrix multiplications in parallel, so training is many times faster; NVIDIA CUDA and cuDNN provide the software layer. TF 2.10 was the last release with native Windows GPU support.

Q11. Describe the MNIST dataset you loaded.

70,000 grayscale images of handwritten digits 0-9, size 28 x 28: 60,000 for training and 10,000 for testing, 10 classes. Pixel values are 0-255.

Q12. TensorFlow or PyTorch: which would you choose?

PyTorch for research and quick experimentation; TensorFlow/Keras when deployment to production, mobile or browser is important. Both are mature and can do the same models.

Q13. Name other deep learning frameworks and the ONNX format.

MXNet, CNTK, Caffe, JAX, Chainer, Deeplearning4j. ONNX is an open format for exchanging trained models between frameworks.

Q14. What is a computational graph? What is TensorBoard?

A graph of operations (nodes) connected by tensors (edges) describing a computation; frameworks use it for gradients and optimisation. TensorBoard is TensorFlow's visualisation tool for loss curves, graphs, histograms and images.

Q15. How did you install the packages? Why is `pip install tensorflow` used inside the notebook?

With `pip` from the notebook cell (`pip install tensorflow`, `keras`, `theano`, `torch`) in an Anaconda/Jupyter environment. The version must match the Python version (TF 2.10 supports Python 3.7-3.10). A kernel restart may be needed after installing.

Practical 2: Feedforward Neural Network with Keras and TensorFlow (MNIST, SGD)

Snapshot of your notebook

Notebook	DL_LAB_ASS_02.ipynb
Architecture	Flatten (784) -> Dense 128 ReLU -> Dense 10 softmax
Parameters	Dense 1: 784x128+128 = 100,480. Dense 2: 128x10+10 = 1,290. Total 101,770, all trainable
Compile	optimizer `sgd`, loss `sparse_categorical_crossentropy`, metric accuracy; 10 epochs, default batch 32
Result	Epoch 1: loss 0.6522, acc 0.8363. Epoch 10: loss 0.1692, acc 0.9521, val_acc 0.9522. Test loss 0.164, test accuracy 0.952
Confusion matrix	9,522 of 10,000 on the diagonal. Largest errors: 4 predicted as 9 (24), 7 as 9 (21), 5 as 3 (19), 7 as 2 (18)

Questions and answers

Q1. Explain the architecture of your network.

Input 28 x 28 image, `Flatten` makes a 784-vector, a hidden Dense layer of 128 ReLU neurons, and an output Dense layer of 10 softmax neurons (one per digit). It is a fully connected feedforward network.

Q2. Why is the Flatten layer used?

Dense layers take 1-D input, so the 2-D image is unrolled to 784 values. It has no parameters (0 in `model.summary()`).

Q3. Why divide pixel values by 255?

To scale pixels from 0-255 to 0-1, which keeps gradients stable and speeds convergence.

Q4. Why ReLU in the hidden layer and softmax in the output layer?

ReLU is cheap and avoids vanishing gradients for positive inputs. Softmax converts the 10 output scores into probabilities that sum to 1; `argmax` gives the predicted digit.

Q5. Sparse categorical vs categorical cross-entropy?

Both compute `-log(probability of the true class)`. `sparse_categorical_crossentropy` accepts integer labels (5, 0, 4...) as in your code; `categorical_crossentropy` needs one-hot labels, e.g. via `to_categorical`.

Q6. How are the parameters counted in `model.summary()`?

Dense layer parameters = inputs x units + units (biases). Layer 1: 784x128+128 = 100,480. Layer 2: 128x10+10 = 1,290. Total 101,770.

Q7. Explain the SGD optimizer and its role in your code.

Stochastic gradient descent updates weights with `w = w - lr * gradient` computed on a mini-batch. Keras default learning rate is 0.01 and batch size 32, so each epoch has 60,000/32 = 1,875 updates (`1875/1875` in the log).

Q8. Why is the accuracy only 95% with SGD? How can you improve it?

Plain SGD with lr 0.01 converges slowly and your loss was still falling at epoch 10 (val_loss 0.164). Improve with Adam, more epochs, more hidden units or layers, dropout, batch normalization, or a CNN (Practical 3 reaches 98.8%).

Q9. Explain backpropagation for this network.

For softmax with cross-entropy the output error is `y_hat - y`. It is multiplied back through the weights `W2^T` and the ReLU derivative to get the hidden-layer error, from which `dL/dW1` and `dL/dW2` are computed; SGD then updates W1, b1, W2, b2.

Q10. What are `loss`, `accuracy`, `val_loss` and `val_accuracy`?

Loss and accuracy are measured on training batches; `val_` versions on the validation data after each epoch. Here validation data is the test set, which is fine for a lab but in practice a separate validation split should be used.

Q11. How do you read your training result? Is there overfitting?

Training accuracy rose 0.836 to 0.952 and validation accuracy 0.904 to 0.952; both are almost equal and both still improving, so there is no overfitting, more likely slight underfitting that more epochs would fix.

Q12. Explain your confusion matrix.

Rows are true digits, columns predicted digits; the diagonal is correct predictions. Sum of the diagonal 9,522 = 95.22% accuracy. Typical confusions are visually similar digits: 4 vs 9, 7 vs 9, 5 vs 3.

Q13. What is the universal approximation theorem?

A feedforward network with one hidden layer and enough neurons with a non-linear activation can approximate any continuous function to any accuracy. Depth makes it more efficient.

Q14. What would happen without the activation function?

The network becomes linear (a single matrix multiplication) and could not separate non-linear classes such as digits.

Q15. How are weights initialised in Keras Dense layers?

Weights: Glorot (Xavier) uniform; biases: zeros. Random initialisation breaks symmetry so neurons learn different features.

Q16. What does `random.randint(0, 9999)` and `plt.imshow` do in your notebook?

Picks a random test image index and displays that digit so you can visually check a sample; with `model.predict` you can compare the predicted label with it.

Q17. Difference between shallow and deep networks?

Shallow: one hidden layer. Deep: several hidden layers, which learn hierarchical features (edges, then shapes, then objects).

Q18. Give some applications of feedforward networks.

Classification and regression on tabular data, digit recognition, credit scoring, simple pattern recognition; they are the basis of all other architectures.

Check before the exam

Missing step: the syllabus step (f) 'Plot the training loss and accuracy' is not in the notebook. Add this cell before the exam: `plt.plot(history.history['loss'], label='loss'); plt.plot(history.history['val_loss'], label='val_loss'); plt.plot(history.history['accuracy'], label='accuracy'); plt.plot(history.history['val_accuracy'], label='val_accuracy'); plt.legend(); plt.show()`

The prediction section only shows a random image; add `print(np.argmax(model.predict(x_test[n:n+1])))` to show the model's prediction for it.

Practical 3: Image Classification with a CNN (MNIST, 4 stages)

Snapshot of your notebook

Notebook	DL_LAB_ASS_03.ipynb
Stages	(a) load and preprocess, (b) define architecture, (c) train, (d) estimate performance
Architecture	Conv2D 32 (3x3, ReLU) -> MaxPool 2x2 -> Flatten -> Dense 100 ReLU -> Dense 10 softmax
Shapes and params	Input 28x28x1 -> 26x26x32 (320 params) -> 13x13x32 -> 5,408 -> Dense 540,900 -> Dense 1,010. Total 542,230
Compile	SGD(learning_rate 0.01, momentum 0.9), sparse categorical cross-entropy, accuracy; 10 epochs
Result	Test loss 0.0498, test accuracy 0.9879 (`accuracy_score` also 0.9879). Model saved as `model_digit.json` + `model_digit.h5`

Questions and answers

Q1. What is a CNN? Why is it better than an MLP for images?

A convolutional neural network uses convolution layers with small shared filters that slide over the image. An MLP on a 224x224x3 image would need 150,528 inputs per neuron and ignores spatial structure; a CNN has far fewer parameters, uses local patterns and is translation-equivariant.

Q2. Explain the convolution operation and the output size formula.

A kernel (e.g. 3x3) slides over the input; at each position the element-wise products are summed to give one value of the feature map. Output size = `(n - f + 2p)/s + 1`. Yours: (28 - 3 + 0)/1 + 1 = 26.

Q3. How many parameters does your Conv2D layer have?

`(3 x 3 x 1 + 1) x 32 = 320`: each of the 32 filters has 9 weights (1 input channel) plus 1 bias.

Q4. What are stride, padding and filters?

Stride = step of the kernel (1 here). Padding: `valid` = none (Keras default, so 28 shrinks to 26), `same` = zero padding that keeps the size. The number of filters (32) = number of feature maps produced, each detecting a different pattern.

Q5. Explain parameter sharing, local connectivity and equivariance.

The same filter weights are reused at every position (parameter sharing), each neuron looks only at a small region (local connectivity). Equivariance: if the input shifts, the feature map shifts by the same amount; pooling adds a little invariance.

Q6. What is pooling? Why max pooling 2x2?

Pooling downsamples a feature map: max pooling keeps the maximum in each 2x2 window (stride 2), so 26x26 becomes 13x13. It cuts computation and parameters, gives small translation invariance and reduces overfitting. It has no learnable parameters (0 in the summary). Average pooling takes the mean.

Q7. Why `Flatten`, and how did you get 5,408?

To feed the 3-D feature maps to Dense layers. 13 x 13 x 32 = 5,408.

Q8. How are 540,900 and 1,010 obtained? Where are most parameters?

Dense 100: 5,408 x 100 + 100 = 540,900. Dense 10: 100 x 10 + 10 = 1,010. Total 542,230; almost all are in the first Dense layer, not in the convolution.

Q9. Why reshape the data to (28, 28, 1)?

Conv2D expects 4-D input (batch, height, width, channels). MNIST is grayscale = 1 channel; RGB images would have 3.

Q10. What do momentum = 0.9 and learning rate 0.01 mean?

Each update combines the current gradient with 90% of the previous velocity, so the optimizer moves faster along consistent directions and oscillates less than plain SGD.

Q11. Map your notebook to the four stages.

(a) Loading and preprocessing: `mnist.load_data()`, scaling to 0-1, reshape to (28,28,1). (b) Architecture: the Sequential model. (c) Training: `model.fit` for 10 epochs. (d) Performance: `accuracy_score`, `model.evaluate`, accuracy and loss curves.

Q12. Interpret your result. How does it compare with Practical 2?

98.79% test accuracy and loss 0.0498 versus 95.2% for the feedforward net, because convolution captures local spatial patterns (strokes, curves) that Flatten+Dense cannot.

Q13. Why did you save the model as JSON and H5?

`model.to_json()` stores only the architecture, `model.save_weights()` stores the learned weights in HDF5. Reload with `model_from_json` and `load_weights`. `model.save()` can store both in one file.

Q14. Name popular CNN architectures.

LeNet-5 (1998, digits), AlexNet (2012: 5 conv + 3 FC, ReLU, dropout, GPU training, won ImageNet), VGG (3x3 conv stacks), GoogLeNet/Inception, ResNet (skip connections), DenseNet (dense connections), MobileNet, EfficientNet.

Q15. What are the basic building blocks of a CNN?

Convolution layers, non-linear activation (ReLU), pooling, flatten, fully connected layers and a softmax output; optionally batch normalization and dropout.

Q16. How would you reduce overfitting in a CNN?

Data augmentation (flip, rotate, zoom), dropout after Dense layers, batch normalization, L2 regularization, early stopping, smaller network.

Q17. Name variants of the basic convolution function.

Strided and dilated (atrous) convolution, transposed convolution (upsampling), depthwise separable convolution (MobileNet), 1x1 convolution (channel mixing), 3-D convolution for video.

Q18. What is a receptive field?

The region of the input image that affects one unit of a feature map; it grows with depth, so deeper layers see larger parts of the image.

Q19. Difference between Conv2D and Dense layers?

Dense connects every input to every neuron with separate weights; Conv2D connects each neuron to a small local window and shares one filter across the whole image.

Check before the exam

Validation anomaly: the training log shows `val_loss` around 23 while the final `model.evaluate` gives loss 0.0498. The cause is `validation_data = (x_test, y_test)` (lowercase) in `model.fit`, but your preprocessed arrays are `X_test` and `y_test`. Also the log shows train accuracy 1.0000 and loss 2e-4 already in epoch 1, which means the cell was run again on an already trained model. If the examiner asks, say so and re-run once from the top with `validation_data=(X_test, y_test)`; you should then see val_loss near 0.05 and val_accuracy near 0.98.

The notebook plots the curves with a hard-coded `epochs = 10`; keep it equal to the epochs in `fit`.

Practical 4: Autoencoder for Anomaly Detection (Credit Card Fraud)

Snapshot of your notebook

Notebook	DL_ASS4.ipynb; dataset `creditcard.csv` (Kaggle European card transactions: 284,807 rows, 492 frauds, about 0.17%)
Preprocessing	Dropped `Time`; `Amount` standardised with StandardScaler; 80/20 split (`random_state=42`); training uses only normal rows (Class 0); input dimension 29
Architecture	Functional API: Dense 14 tanh (L1 activity regularizer 1e-4) -> Dense 7 ReLU (bottleneck) -> Dense 7 tanh -> Dense 29 ReLU. Parameters: 420+105+56+232 = 813
Compile and train	Adam, MSE loss, metric accuracy, batch 32, up to 100 epochs, EarlyStopping (val_loss, patience 10) stopped at epoch 72, best weights from epoch 62; ModelCheckpoint saved `model.h5`
Error statistics	Test set 56,962 transactions: mean error 0.7367, median 0.3826, maximum 260.99. Threshold chosen: 50
Result	Accuracy 0.9984, recall 0.2551, precision 0.5682. Implied counts: about 98 frauds in the test set, TP 25, FP 19, FN 73, TN 56,845

Questions and answers

Q1. What is an autoencoder? Explain encoder, latent space and decoder.

A neural network trained to reproduce its own input (unsupervised, target = input). The encoder compresses the input `x` into a low-dimensional latent code `z` (the bottleneck, 7 neurons here); the decoder reconstructs `x^` from `z`. The loss is the reconstruction error `||x - x^||^2`.

Q2. What is an undercomplete autoencoder?

One whose code dimension is smaller than the input (29 to 7 here), which forces it to learn only the most important structure of the data instead of copying the input.

Q3. Types of autoencoders (syllabus)?

Undercomplete; regularized/sparse (penalty on activations, as your `activity_regularizer=l1`); denoising (corrupt the input, reconstruct the clean one); contractive (penalty on the Jacobian of the encoder); stochastic/variational (encoder outputs a distribution, KL term); stacked (deep) and convolutional autoencoders.

Q4. How does an autoencoder detect anomalies? Why train on normal data only?

It learns to reconstruct normal transactions well. A fraudulent transaction looks different, is reconstructed badly and gets a high reconstruction error. If error > threshold, flag it. That is why `X_train` keeps only `Class == 0`.

Q5. How did you compute the reconstruction error?

`mse = np.mean(np.power(X_test - predictions, 2), axis=1)`: squared difference per feature, averaged over the 29 features for each transaction.

Q6. How is the threshold selected? Is 50 a good choice?

You picked 50 by looking at the error plot (normal errors mostly below 1, a few frauds up to 261). Better methods: a high percentile of normal errors (e.g. 99th), mean + k x std, or the value that maximises F1 on the precision-recall curve. Lowering the threshold raises recall but lowers precision.

Q7. Interpret accuracy 99.84%, recall 25.5% and precision 56.8%.

Fraud is only 0.17% of data, so a model that labels everything normal already scores about 99.83% accuracy; accuracy is misleading here. Recall 25.5% means only about a quarter of frauds are caught; precision 56.8% means about 57% of alerts are real fraud. F1 = 2PR/(P+R) is about 0.35.

Q8. What is class imbalance and how can it be handled?

One class is far rarer than the other. Use precision, recall, F1, PR-AUC or ROC-AUC instead of accuracy; resampling (oversampling/SMOTE, undersampling); class weights; or anomaly-detection methods like this one.

Q9. Why was `Time` dropped and `Amount` standardised?

`Time` (seconds since first transaction) carries little useful signal; `Amount` has a much larger range than the PCA features V1-V28, so it is scaled with `z = (x - mean)/std` to similar magnitude.

Q10. Why is `metrics=['accuracy']` not meaningful here?

Accuracy compares predicted and true labels exactly; an autoencoder outputs continuous values, so the reported 0.74 does not measure quality. The loss (MSE) is the real indicator.

Q11. Why tanh, ReLU and L1 regularization in your model?

tanh/ReLU add non-linearity. The L1 activity regularizer (1e-4) penalises large activations so only a few neurons fire, giving a sparse code. A linear output activation is more common for reconstructing standardized data, because ReLU cannot output negative values.

Q12. Explain EarlyStopping and ModelCheckpoint.

EarlyStopping monitors `val_loss` and stops after 10 epochs without improvement of at least 0.0001, restoring the best weights (it stopped at epoch 72 and restored epoch 62). ModelCheckpoint saves the best model to `model.h5`. The TensorBoard object was created but not passed to `callbacks`, so no logs were written.

Q13. What is the relation between PCA and an autoencoder?

A linear autoencoder with MSE loss learns the same subspace as PCA. With non-linear activations and several layers an autoencoder can capture non-linear structure and is more powerful.

Q14. Applications of autoencoders?

Dimensionality reduction, denoising images, anomaly and fraud detection, image compression, feature learning and pre-training, recommendation, and (variational/ generative versions) data generation.

Q15. How many steps per epoch and why 7,108?

Steps = training rows / batch size. 7,108 x 32 is about 227,450 normal training transactions (80% of the data without fraud rows), batch size 32.

Q16. What does `shuffle=True` and `random_state=42` do?

`shuffle=True` reshuffles training data every epoch to avoid order bias; `random_state=42` makes the train/test split reproducible.

Q17. How does your confusion matrix look and what do FP and FN mean here?

FP: a normal transaction flagged as fraud (customer inconvenience); FN: a fraud that was missed (financial loss). In your run roughly 19 FP and 73 FN.

Q18. Which loss does the autoencoder use and why not cross-entropy?

Mean squared error, because the targets are real-valued features; cross-entropy suits class probabilities.

Check before the exam

Be ready to explain the low recall (25.5%). Honest answer: threshold 50 is very strict, the reconstruction MSE is high (about 0.70) so the model is not reconstructing well, and a ReLU output cannot produce negative values. Mention improvements: choose the threshold from a percentile or the PR curve, linear output layer, a validation set of only normal data.

Include the numbers: threshold 50, accuracy 0.9984, recall 0.2551, precision 0.5682.

Practical 5: Continuous Bag of Words (CBOW) Model

Snapshot of your notebook

Notebook	DL_ASS5.ipynb; text file `corona.txt` (lines with at least 2 spaces kept)
Data preparation	`Tokenizer` builds word index; text converted to integer sequences; 198 tokens in total; window size 2 (4 context words)
Model	Embedding(output 100) -> Lambda(mean over context) -> Dense softmax over the vocabulary; loss categorical cross-entropy, optimizer Adam
Output	Embedding weights written to `vectors.txt` in word2vec text format; loaded with gensim `KeyedVectors`; `cbow_output['the'].shape` = (100,)

CBOW vs Skip-gram

	CBOW	Skip-gram
Task	Predict the centre word from its context words	Predict the context words from the centre word
Speed	Faster (one prediction per window)	Slower (several predictions per word)
Best for	Frequent words, small windows	Rare words, small data sets
Input / output	Many words -> one word	One word -> many words

Questions and answers

Q1. What is a word embedding? Why not one-hot vectors?

A dense, low-dimensional real-valued vector (here 100 numbers) in which similar words lie close together. One-hot vectors are as long as the vocabulary, sparse, and every pair of words is equally distant, so no meaning is captured.

Q2. What is Word2Vec?

A shallow two-layer network by Mikolov et al. (Google, 2013) that learns word embeddings from raw text. It has two architectures: CBOW and Skip-gram.

Q3. Explain the CBOW model.

Continuous Bag of Words predicts the target (centre) word from the surrounding context words. With window size 2, the 2 words before and 2 after the target are input; their embeddings are averaged (order ignored, hence 'bag') and a softmax layer predicts the target word.

Q4. Describe the architecture of your model.

`Embedding(V, 100)` looks up a 100-dim vector for each of the 4 context words; `Lambda(K.mean, axis=1)` averages them to one 100-dim vector; `Dense(V, softmax)` outputs a probability for every vocabulary word. The embedding matrix after training is the word-vector table.

Q5. What is the embedding layer?

A trainable lookup table of shape (vocabulary size, embedding dimension). Row `i` is the vector of word `i`. It is equivalent to multiplying a one-hot vector by a weight matrix.

Q6. What is the loss function and training objective?

Maximise the probability of the true centre word given the context, i.e. minimise categorical cross-entropy `-log P(w_t | context)`. The full softmax over a large vocabulary is expensive, so real Word2Vec uses negative sampling or hierarchical softmax.

Q7. What does the window size mean? How are training pairs generated?

Window size = number of words taken on each side of the target. For each word in a sentence the code collects up to 2 words before and 2 after (`range(begin, end)`, skipping the word itself) as input and the word itself as the label; shorter contexts at sentence edges are padded with zeros by `pad_sequences`.

Q8. What does Keras `Tokenizer` do?

`fit_on_texts` builds a `word_index` dictionary (word -> integer starting from 1, lower-cased, punctuation removed); `texts_to_sequences` converts sentences into lists of those integers. Index 0 is reserved for padding, so vocabulary size = `len(word_index) + 1`.

Q9. What is cosine similarity? What does `most_similar` do?

`cos(a,b) = (a . b)/(|a||b|)`, the angle between two vectors (1 = same direction). `most_similar(positive=['the'])` returns the words whose vectors have the highest cosine similarity to the vector of 'the'. Analogies work by vector arithmetic, e.g. king - man + woman is close to queen.

Q10. CBOW vs Skip-gram?

See the table: CBOW is faster and good for frequent words; Skip-gram represents rare words better.

Q11. Which hyperparameters matter for Word2Vec?

Embedding dimension (100), window size (2), number of epochs, minimum word count, number of negative samples, learning rate.

Q12. Applications of word embeddings?

Input features for text classification, sentiment analysis, machine translation, search and recommendation, named entity recognition, clustering of similar words, question answering.

Q13. Other embedding methods and limitations of Word2Vec?

GloVe (uses global co-occurrence counts), FastText (character n-grams, handles unseen words), ELMo and BERT (contextual embeddings). Word2Vec gives one fixed vector per word, so it cannot separate word senses (bank of a river vs bank), cannot represent unseen words and needs a large corpus.

Q14. What is the word2vec text file format you wrote?

First line `vocab_size dimensions`; then one line per word: the word followed by its D numbers. Gensim reads it with `KeyedVectors.load_word2vec_format(path, binary=False)`.

Q15. Why `to_categorical(target, total_vocab)`?

Converts the target word index into a one-hot vector of length V to match the softmax output and categorical cross-entropy loss.

Q16. What is `pad_sequences`?

It makes all context lists the same length (`maxlen`) by padding with zeros, needed because words near the start or end of a sentence have fewer than 4 neighbours.

Check before the exam

Your output is not valid, and the examiner may notice. The training cost prints `0` for every epoch and `most_similar` returns number-like tokens (`111213`, `.007863175`). Reasons:

(1) `cbow_model(data, ...)` is given `data`, the file object that was already read to the end, so the generator yields nothing, the training loop never runs, cost stays 0 and the embeddings remain random. It must be `corona_data`.

(2) The loop uses `contextual, final_target` instead of the yielded `x, y`.

(3) `total_vocab = sum(len(s) ...)` counts all tokens (198), not the vocabulary. Use `len(vectorize.word_index) + 1`.

(4) `pad_sequences(..., total_length=...)` should be `maxlen=`.

(5) The vectors are written as text but loaded with `binary=True`; it must be `binary=False`. The header should contain the number of words written.

Corrected version (same structure, same old Keras API as your Keras 2.10 environment; I could not run TensorFlow here, only checked the generator logic with a small mock): see the code block below.

```
import numpy as np
import keras.backend as K
from keras.models import Sequential
from keras.layers import Dense, Embedding, Lambda
from keras.utils import to_categorical
from keras.preprocessing import sequence
from keras.preprocessing.text import Tokenizer
import gensim
data=open('D:\\corona.txt','r')
corona_data=[text for text in data if text.count(' ')>=2]
vectorize=Tokenizer()
vectorize.fit_on_texts(corona_data)
corona_data=vectorize.texts_to_sequences(corona_data)
total_vocab=len(vectorize.word_index)+1
window_size=2
def cbow_model(data,window_size,total_vocab):
    total_length=window_size*2
    for text in data:
        text_len=len(text)
```

```
        for idx,word in enumerate(text):
            begin=idx-window_size
            end=idx+window_size+1
            context_word=[[text[i] for i in range(begin,end) if 0<=i<text_len and i!=idx]]
            contextual=sequence.pad_sequences(context_word,maxlen=total_length)
            final_target=to_categorical([word],total_vocab)
            yield contextual,final_target
model=Sequential()
model.add(Embedding(input_dim=total_vocab,output_dim=100,input_length=window_size*2))
model.add(Lambda(lambda x:K.mean(x,axis=1),output_shape=(100,)))
model.add(Dense(total_vocab,activation='softmax'))
model.compile(loss='categorical_crossentropy',optimizer='adam')
for i in range(10):
    cost=0
    for x,y in cbow_model(corona_data,window_size,total_vocab):
        cost+=model.train_on_batch(x,y)
    print(i,cost)
dimensions=100
vect_file=open('D:/vectors.txt','w')
vect_file.write('{} {}\n'.format(total_vocab-1,dimensions))
weights=model.get_weights()[0]
for text,i in vectorize.word_index.items():
    vect_file.write('{} {}\n'.format(text,' '.join(map(str,weights[i,:]))))
vect_file.close()
cbow_output=gensim.models.KeyedVectors.load_word2vec_format('D:/vectors.txt',binary=False)
print(cbow_output.most_similar(positive=['the']))
```

Practical 6: Transfer Learning with a Pre-trained CNN (VGG16, tf_flowers)

Snapshot of your notebook

Notebook	Assignemnt_6.ipynb; CPU only (`CUDA_VISIBLE_DEVICES=-1`), about 90 s per epoch
Data	`tf_flowers` (3,670 images, 5 classes). 70% = 2,569 training images; images resized to 150 x 150; labels one-hot
Pre-trained model	VGG16, `weights='imagenet'`, `include_top=False`, `trainable=False` (14,714,688 frozen parameters), `preprocess_input` applied
Custom classifier	Flatten (4x4x512 = 8,192) -> Dense 50 ReLU (409,650) -> Dense 20 ReLU (1,020) -> Dense 5 softmax (105). Trainable 410,775; total 15,125,463
Training	Adam, categorical cross-entropy, validation_split 0.2, batch 32, EarlyStopping (val_accuracy, patience 5, restore best): stopped after epoch 12, best epoch 7 (val_acc 0.7257)
Result	Epoch 12: train accuracy 0.9912, val_accuracy 0.7121, val_loss 1.35. Test loss 0.1068, test accuracy 0.9691

When to freeze or fine-tune (dataset size vs similarity to ImageNet)

	Similar to ImageNet	**Different from ImageNet**
Small dataset	Freeze base, train only new classifier (your case)	Use lower-layer features, train more of the top layers
Large dataset	Fine-tune the top conv blocks with a small learning rate	Fine-tune many layers or train from scratch

Questions and answers

Q1. What is transfer learning? Why use it?

Re-using a network trained on a large dataset (ImageNet) as the starting point for a new task. It needs less data and training time and usually gives better accuracy because early layers already detect general features (edges, textures), later layers more specific parts and objects.

Q2. Feature extraction vs fine-tuning?

Feature extraction: freeze the pre-trained base and train only the new classifier head (what your notebook does). Fine-tuning: afterwards unfreeze some top conv layers and train them with a very small learning rate (e.g. 1e-5) to adapt the features to the new data.

Q3. Explain the five syllabus steps and where they are in your code.

(a) Load VGG16 with ImageNet weights, (b) `base_model.trainable = False` freezes the conv layers, (c) Flatten + Dense 50 + Dense 20 + Dense 5 added with `Sequential`, (d) `model.fit` trains only these layers, (e) fine-tuning: unfreeze later layers and lower the learning rate. Step (e) is not in the notebook; to add: `base_model.trainable=True`, freeze early blocks, `model.compile(optimizer=Adam(1e-5), ...)`, `fit` for a few epochs.

Q4. Describe the VGG16 architecture.

16 weight layers: 13 convolution layers (all 3x3, stride 1, same padding, ReLU) in 5 blocks separated by 2x2 max pooling, followed by 3 fully connected layers (4096, 4096, 1000 softmax). About 138 million parameters; the conv base alone has 14,714,688, as shown in your summary. Developed by the Visual Geometry Group, Oxford (2014).

Q5. What does `include_top=False` mean?

It removes the original three fully connected layers and the 1000-class output, leaving only the convolutional base, so you can attach your own classifier and use a different input size (150 x 150 instead of 224 x 224).

Q6. What is ImageNet?

A dataset of over a million labelled images in 1,000 classes (ILSVRC) used to pre-train the models.

Q7. What does `preprocess_input` do for VGG16?

Converts RGB to BGR and subtracts the ImageNet mean of each channel (no division by 255). The input must be processed exactly as during the original training.

Q8. How did you get the numbers 8,192, 409,650, 1,020 and 105?

After 5 poolings 150 becomes 75, 37, 18, 9, 4, so the output is 4 x 4 x 512 = 8,192. Dense 50: 8,192 x 50 + 50 = 409,650. Dense 20: 50 x 20 + 20 = 1,020. Dense 5: 20 x 5 + 5 = 105.

Q9. Why one-hot labels with categorical cross-entropy? Why softmax with 5 units?

5 mutually exclusive flower classes, so the output is a probability distribution (softmax) and the labels are one-hot (`to_categorical`). With integer labels use the sparse version.

Q10. Explain EarlyStopping with your settings.

`monitor='val_accuracy', mode='max', patience=5, restore_best_weights=True`: training stops when validation accuracy has not improved for 5 epochs and the weights of the best epoch (epoch 7) are restored. Training stopped after epoch 12.

Q11. What does `validation_split=0.2` do?

Keras takes the last 20% of the training arrays (before shuffling) for validation: here about 514 images, leaving 2,055 for training (65 steps of batch 32).

Q12. Interpret your training curves. Is the model overfitting?

Yes. Training accuracy climbs to 99% while validation accuracy stays at about 70-72% and validation loss rises from 0.94 (epoch 4) to 1.35. Fixes: data augmentation, dropout, `GlobalAveragePooling2D` instead of Flatten, fewer trainable parameters, fine-tuning with a small learning rate.

Q13. Why is the optimizer Adam?

Adam adapts the step size per parameter using moving averages of gradients and squared gradients; it converges quickly with default lr 0.001 and little tuning.

Q14. Object detection vs image classification? (the syllabus title says object detection)

Classification gives one label for the whole image (what). Detection finds every object, with a bounding box and class (what and where). Two-stage detectors: R-CNN, Fast R-CNN, Faster R-CNN. One-stage: YOLO, SSD, RetinaNet. Key terms: bounding box, IoU (intersection over union), anchor boxes, non-maximum suppression, mAP. Detectors also use a pre-trained CNN (VGG, ResNet) as backbone; your notebook applies the same transfer-learning steps to classification.

Q15. Other pre-trained models you could use?

ResNet50 (skip connections), InceptionV3, MobileNet (light, depthwise separable convolution), EfficientNet (compound scaling), DenseNet.

Q16. What is DenseNet? (syllabus Unit V)

Each layer receives the feature maps of all previous layers and passes its own to all later ones. Benefits: feature reuse, fewer parameters, stronger gradient flow (less vanishing gradient).

Q17. What is domain adaptation? What is greedy layer-wise pre-training?

Domain adaptation: adapting a model trained on a source domain to a different but related target domain. Greedy layer-wise pre-training: train one layer (e.g. an autoencoder) at a time, unsupervised, then stack and fine-tune; it was the early way to train deep networks.

Q18. What does `tfds.load(..., batch_size=-1, as_supervised=True)` do?

Loads the whole split as a single batch of tensors in (image, label) form; `split=['train[:70%]', ...]` selects slices of the dataset.

Q19. Why resize all images to 150 x 150?

The images have different sizes (e.g. 442 x 1024); a batch needs equal shapes, and a smaller size reduces computation.

Check before the exam

Data leakage in your test set. You used `split=['train[:70%]', 'train[:30%]']`, so the 'test' images (first 30%) are a subset of the training images (first 70%). That is why test accuracy is 96.9% while validation accuracy is only about 72%. The correct split is `['train[:70%]', 'train[70%:]']`; expect a test accuracy close to the validation accuracy (around 70-75%). If asked, say the 96.9% is inflated because the model has seen those images.

The accuracy plot shows only the training curve; add `plt.plot(history.history['val_accuracy'])` and use legend `['train','val']`.

The fine-tuning step (e) is missing; be ready to describe it (see answer 3).